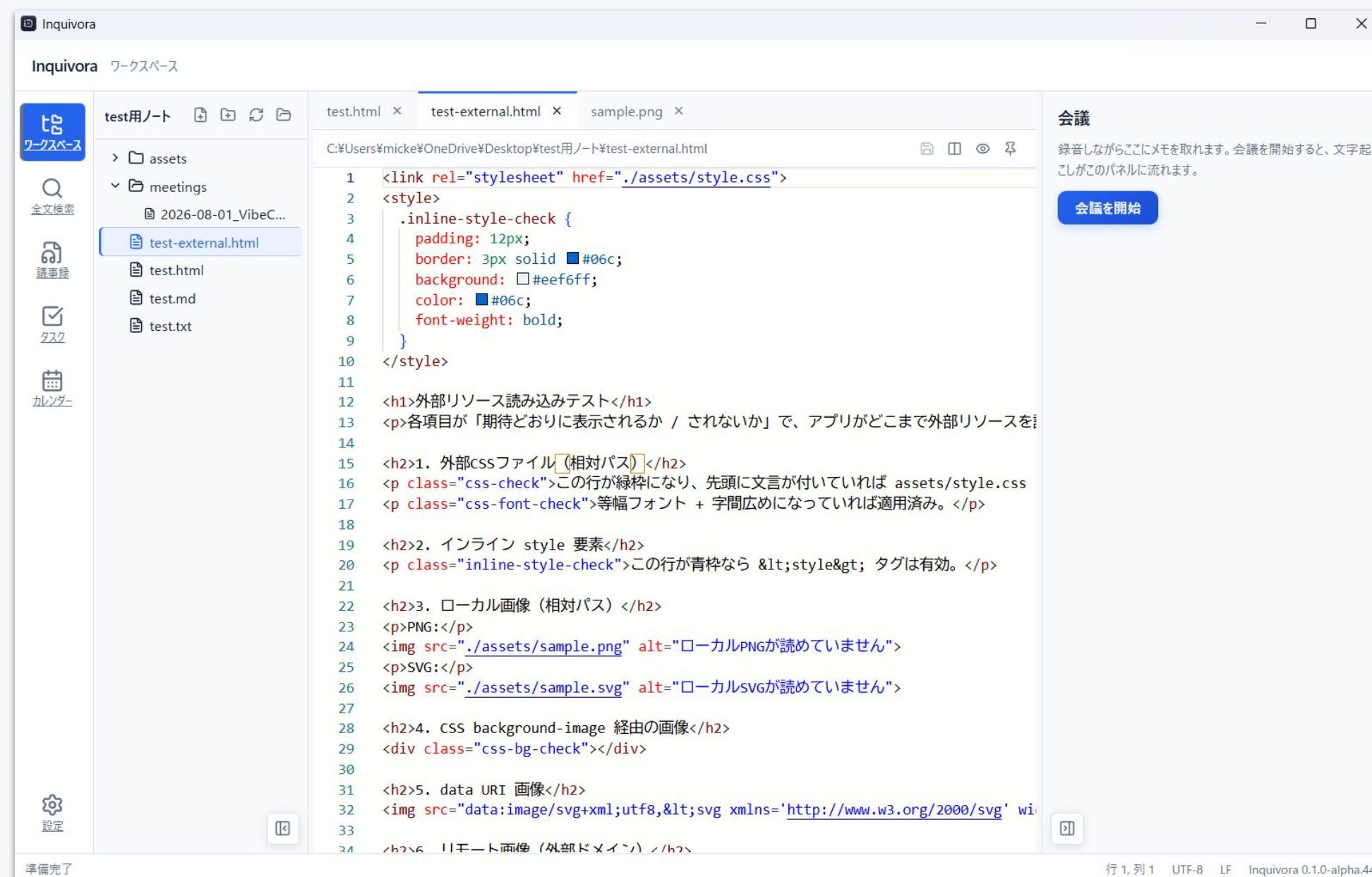


# バイブコーディングで メモアプリ作成

Inquivora — メモ・会議・タスクをつなぐ  
ローカルファーストの統合ワークスペース



# 目次

01. 自己紹介

---

02. 作ったもの (Inquivora)

---

03. 技術構成

---

04. 開発の進め方

---

05. 開発しやすくなった5つの工夫

---

06. 仕様書を Markdown に

---

07. 失敗① UIの作成

---

08. 失敗② API仕様

---

09. 未解決の課題

---

10. 学びと今後

---

# 02. 作ったもの

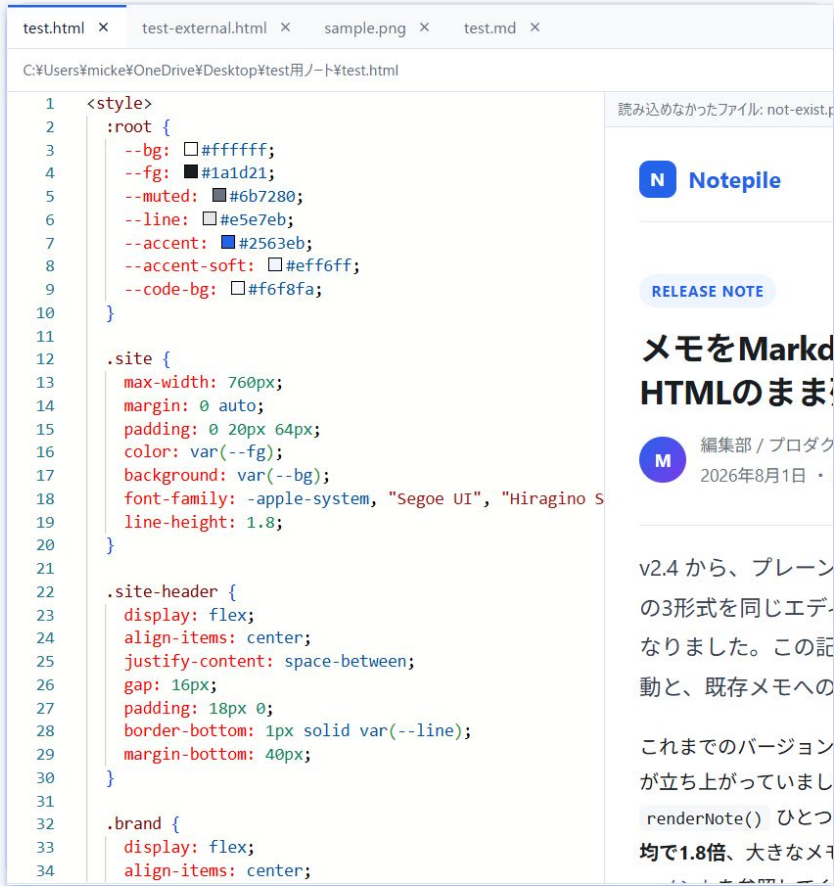
Inquivora：メモ・会議・タスクをつなぐ

メモ帳アプリを作成しました。ただ機能の数が多いだけでなく、情報をつなげることをメインに作成。

- 1
- メモ（Markdown）
- 2
- メモ（HTML）
- 3
- 会議 → AI議事録
- 4
- カレンダー



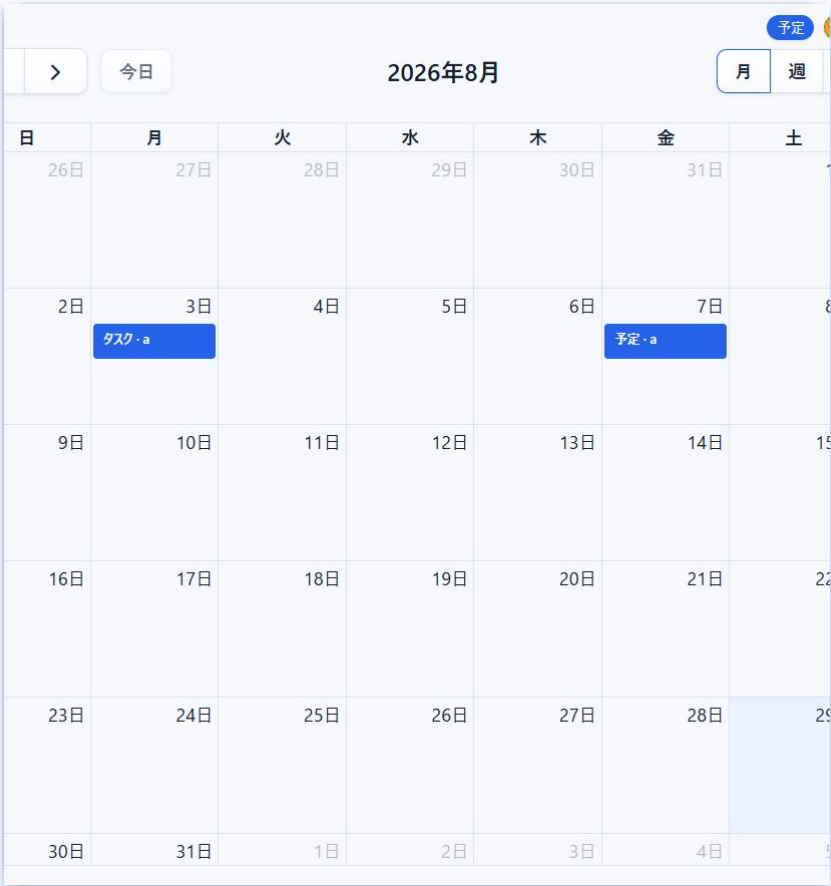
書きながら右側でプレビュー



HTML もそのまま表示



文字起こし・要約からタスク化



予定・タスクが見える化

『メモ・会議・タスク管理・カレンダー』をこのアプリ一つで完結

# 03. 技術構成

4つの層で構成し、テストとデプロイは GitHub Actions に任せた

TS

## React / TypeScript

フロントエンド（UI・画面遷移）

Rs

## Tauri 2 / Rust

アプリケーション層

C#

## .NET 8 / C# Sidecar

音声・ネイティブ機能（NAudio / Whisper.net 等）

DB

## SQLite / FTS5

データ層（全文検索インデックス）

CI / CD

## GitHub Actions

テストとデプロイの自動化を任せ、開発効率と信頼性を確保

## AIとの協調による開発推進

バンプコーディングでAIに任せた部分も多く、  
今は完全には説明しきれない部分もあります。

GitHub Actionsによる自動テストやデプロイを構築しつつ、  
完全に把握しきれていない挙動は、AIに実行・説明させることで主  
体的な理解を深めています。

# 04. 開発の進め方

1 発で完成させるのではなく、何度も下記のループを繰り返しました。



# 05. 開発しやすくなった5つの工夫

全部ルールではなく、今の自分に合っていた進め方

**1** 仕様を Markdown に言語化し、変更時も更新する

**2** 機能単位で小さく実装し、動作確認できる状態を保つ

**3** 関連する修正を 3〜4点程度に絞る

**4** 実装AIとは別のAIにレビューを依頼し、  
評価・テストを行う  
別AIだから必ず正しいわけではない。  
最終的には操作・テスト・ログ・公式仕様で確認する

**5** AIへの伝え方を工夫する

**状況を整理してから、根拠と一緒に伝える**

一度AI自身にエラーが起きた理由を考えさせてから、その内容を修正するよう実行させる

**プロセス** 診断と修正を分ける

- エラーメッセージ・ログ・関係コードを確認させる
- 原因の候補と、その根拠を説明させる
- 修正方針と影響範囲を出させる

**💡 導入効果**

勝手な行動を抑制でき、かつ人間側の原因理解も深まるため、よりの確な指示出しが可能になります。

# 06. 仕様書を Markdown に

AIと人間が参照するための仕様書を作成し開発を行う

## メリット

- ・ AIを切り替えても同じ文脈を共有できる
- ・ 後から仕様や判断の経緯を確認できる
- ・ 変更時にコードと仕様をセットで更新できる
- ・ 検索・修正がしやすい

## デメリット

- ・ 仕様書が古いと、AIへ誤った指示を与える危険がある

ポイント：README とは別に、目的・機能・制約・完了条件をまとめた仕様書を1つ作る

M↓ Inquivora\_実装仕様書.md

### Inquivora 実装仕様書

Inquivora — すべてを取り込み、ひとつにつなぐ。  
Windows向け・ローカルファースト統合ワークスペース

#### 0. 文書情報

| 項目      | 内容                                 |
|---------|------------------------------------|
| 文書名     | Inquivora 実装仕様書                    |
| 対象バージョン | MVP 0.1.0                          |
| 仕様書改訂   | v1.3 (セキュリティ強化・メモ連携タスク・色分けカレンダー対応) |
| 対象OS    | Windows 11 x64                     |
| 配布形式    | NSISセットアップEXE                      |
| データ方針   | ローカルファースト                          |
| AI方針    | BYOK (ユーザー所有APIキー)・必要な処理だけ外部APIへ送信 |
| UI言語    | 日本語                                |
| タイムゾーン  | Asia/Tokyo                         |

本書は、Claude Code、Codex、GitHub Copilot、または開発者へ渡し、そのまま実装へ着手できる粒度を目標とする。

🔒 GitHub で管理し、Claude Code / Codex の双方が参照する

# 07. 失敗① UIの作成

『左右に分割できる』だけでは、実際の操作が足りなかった

## BEFORE

当初の仕様・指示

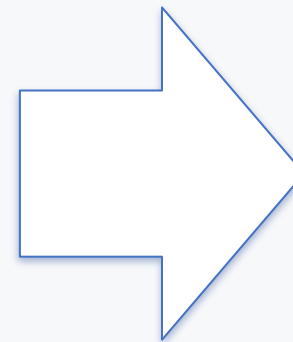
### 最初の指示

「左右に分割してメモを取れる機能を実装して」

### 発生した課題

- ① 一度配置すると入れ替えられない
- ② 左に置いたファイルを右に移動できない
- ③ 右に置いたファイルを別のファイルに置き換えられない

操作性の不足が  
発覚



## AFTER

改善後の仕様・アプローチ

### 課題をまとめてAIに指示し修正 (操作シナリオを提示)

- ・ドラッグで配置を入れ替えられる機能
- ・左右を切り替えられる機能
- ・利用者が直感的に扱えるよう、機能だけでなく「目的」も伝える

### 実現した機能

- ① ファイルをドラッグ&ドロップで配置
- ② 左右どちらのペインにも自在に配置可能
- ③ 左右のペインを直感的に入れ替え可能

画面を作ってもらうときは、見た目だけでなく『操作シナリオ』まで伝える

## 08. 失敗② API仕様

AIの記憶に頼らず、公式仕様・エラーログ・実装を確認して原因を特定した

### 問題

#### 片方の Provider でしか要約が動かなかった

何度か修正するよう依頼したが、  
AIがそれぞれのAPIの仕様に合わせて作ってくれない

→原因や仕様を確認せず、修正だけを繰り返していた

### 解決

#### Provider ごとにリクエスト・レスポンス処理を分け、APIリクエスト確認テストを追加

一度AI自身に公式ドキュメントの情報で  
それぞれのAPIの仕様を調べるよう指示

→整理した仕様に基づいて修正を依頼

API連携のように仕様差分が大きいものや、**修正がうまくいかない場合は**、  
すぐにAIへ修正を依頼するのではなく、現在の公式仕様やエラーログ、実際のコードを確認させ、  
**原因や差分を整理したうえで修正させる。**

# 09. 未解決の課題

## ① 一部の操作で重くなる・少し固まる

### 考えられる原因（仮説）

- ・ ファイル数が多いワークスペースの走査と全文検索インデックス作成
- ・ ローカル Whisper の推論負荷
- ・ 複数の処理が重なった場合のUIへの影響

### 現在の対策

- ・ 検索対象は原則 2MB 以下のファイル
- ・ 128件単位のバッチ処理＋バックグラウンド処理
- ・ 除外フォルダの設定 ・ ローカル Whisper はモデルを一度ロードし、常駐 Sidecar で再利用

トレードオフ：ローカル検索・ローカル文字起こしは Inquivora の核。機能を削れば解決するわけではなく、やりたいことを残しながら待ち時間を減らしたい。

## ② 文字起こし精度

### 影響する要因

- ・ 使用する Whisper モデル
- ・ 音声品質（マイク / 録音環境）
- ・ マイクとPC音声の混在 ・ 話者の重なり
- ・ ノイズの有無
- ・ PC性能（CPU / メモリ / GPU）

条件を固定した検証がまだ十分ではなく、どこまで改善できるのかも現時点では分かっていません。

# 10. 学びと今後

## 学び（この1か月で感じたこと）

- ①AI前提の開発姿勢  
AIは形にしてくれるが、要件・制約・責任は人が持つ
- ②仕様書を残す重要性  
AIを切り替えても同じ方向で開発を続けられる
- ③小さく作って動かす  
一気に完成させず、最小単位で確認する
- ④原因調査と修正を分ける  
ログ・公式仕様・テストを見てから直す
- ⑤最終判断は人間  
AIの成果物をそのまま信じず、自分で確認する

## 今後やりたいこと

- ①非同期処理・通信の理解  
Tauri / Rust / C# 側の処理を深く理解する
- ②コードリーディング力の向上  
AI生成コードを自分で読んで説明できるようにする
- ③感覚ではなく計測で判断  
プロファイリングやログを使って改善する
- ④文字起こし精度の定量評価  
条件を揃えたデータでWhisperの精度を検証する

## まとめ

バイブコーディングで大切なのは、うまいプロンプトを一発で書くことではなく、  
**仕様をしっかりと固めながら作り続けること、そして粘り強く最後まで完成させること**だと感じました。